NutriGent

An agentic diet and allergy assistant for Telegram

Project report — features, architecture, workflows, testing, deployment and roadmap

Author: Kiana Alian
Date: 13 October 2026

Abstract. NutriGent is a self-hosted assistant on Telegram that checks whether foods are safe for a user's allergies, plans meals, builds pantry-aware shopping lists and tracks calories, answering with images rather than long text. It combines a large-language-model agent (Hermes Agent running OpenAI GPT-5.5) with a deterministic Python engine and an SQLite database. Its central principle is *the model proposes, deterministic code disposes*: the model only extracts ingredients, while an auditable rule-based safety gate issues every verdict and fails closed. The system was designed as 48 features across safety, planning, tracking, pantry, autonomy, visuals, adaptation and integrations. This report explains how the system works end to end, describes each workflow, reports honestly which features are working (18), partial (8) or on the roadmap (22), presents the engineering work of the current phase — including the discovery and fix of a fail-open defect and a new step-by-step sign-up — and the evidence from 38 automated tests and an 80-action end-to-end test against a simulated Telegram API. It ends with the deployment, limitations and a six-phase roadmap.

Contents

1. Introduction
2. Why an agent, not a chatbot
3. System architecture
4. How it works: the workflows
5. The deterministic safety gate
6. Feature catalogue
7. User interface and visual replies
8. Nutrition model
9. Data model
10. The agent layer: Hermes Agent and skills
11. Engineering process and fixes in this phase
12. Testing and evidence
13. Deployment
14. Privacy, safety and ethics
15. Current limitations
16. Roadmap
17. Conclusion

Appendices: repository layout, running the system

1. Introduction

People with food allergies make many small safety decisions every day: reading labels, spotting hidden ingredients (whey and casein are milk; Worcestershire sauce contains anchovy; E1105 is egg), planning meals that respect their allergies, preferences, budget and time, and keeping a healthy calorie balance. Existing diet apps tend to fail in four ways, which NutriGent was designed to address:

Problem	How typical apps fail	NutriGent's design answer
Reactive	Nothing happens until the user opens the app	The assistant comes to the user in Telegram and is designed to send plans, reminders and reports on a schedule
Siloed	The barcode scanner does not know the meal plan; the diet app does not guard allergens	One profile and one safety gate behind every feature
No pantry awareness	Recipes ignore what is at home or about to expire	A pantry model; shopping list = what the plan needs minus what is at home
No learning	Every week starts from zero	Preference events and an agent memory designed to improve plans over weeks

Table 1. The problems NutriGent targets.

General-purpose chatbots could help with planning, but they can give confident wrong answers, and for an allergic user a wrong “safe” can mean anaphylaxis. NutriGent therefore pursues four goals:

- **G1 Safety first.** Never declare a food safe when the evidence is insufficient: fail closed.
- **G2 Low friction.** Live in Telegram, prefer tappable buttons to typed commands, answer with images.
- **G3 Personalisation.** Calorie and macro targets, weekly plans, shopping lists, progress charts.
- **G4 Privacy.** Self-hosted: health data stays in a local database on the operator's own server.

The intended users are people with one or more (especially severe) food allergies, people who find weekly meal planning hard, people working towards a calorie or macro target, and small households cooking together. The current deployment is a pilot for 3–10 users.

2. Why an agent, not a chatbot

A chatbot waits for a message and answers it. An agent owns goals, keeps state about the world, acts on its own and improves over time. NutriGent was designed against five agent criteria. Table 2 states how far each one is realised today.

Criterion	Design	Status today
Acts without being asked	Scheduled jobs: daily meal preview and recipe, Saturday shopping reminder, Sunday plan and report, pantry sync four times a week	**roadmap** — skills written, schedule not yet registered
Owns a goal it defends	Weekly calorie target from biometrics; plans aim at it; missed days are rebalanced	**partial** — targets computed and charted; rebalancing on the roadmap
Manages state in the world	Pantry, meal schedule, shopping list and food log are live state	**working** — all stored in SQLite and updated by the bot

Criterion	Design	Status today
Memory that improves behaviour	Preference events distilled into the agent's long-term memory, read before planning	**partial** — agent memory works (restored with 947 messages); the preference loop is not yet wired
Cannot hallucinate a verdict	Deterministic safety gate decides; the model only extracts	**working** — 21 automated safety tests

Table 2. Agent criteria and their current status.

3. System architecture

NutriGent is written in Python (about 1,500 lines of application code and 700 lines of tests). It is split into small modules with one responsibility each (Table 3); Figure 1 shows the deployed architecture.

Component	Responsibility
app/bot.py	Telegram long-polling loop and router. Applies the allow-list, sends users who are mid-sign-up to the sign-up, maps button-grid labels to commands, and dispatches commands, button taps, photos and text. No public port or webhook is needed.
app/onboarding.py	The sign-up conversation: one question at a time; current step and answers stored in the onboarding_state table.
app/handlers.py	One function per user action (check, today, week, month, plan, shopping, log, tired, symptom, report, card, profile).
app/llm.py	The only module that calls OpenAI: image extraction, dish decomposition, weekly planning. Offline sample answers when no key is set.
shared/safety_gate.py	Deterministic verdict engine (Section 5).
shared/nutrition.py	Mifflin–St Jeor targets and macro split (Section 8).
shared/visuals.py, charts.py, reporting.py	Matplotlib renderers for cards and charts, and the bridge from database rows to charts.
shared/telegram_io.py, telegram_menu.py	Minimal Telegram API client (standard library only) and every keyboard: the persistent grid, the main menu and the sign-up buttons.
shared/db.py, schema.sql, allergen_seed.sql	SQLite access, the 13-table schema, the allergen synonym and false-positive seed; idempotent migrations at start-up.
Hermes Agent + skills/	Conversational layer: persona, long-term memory, multilingual chat, scheduled jobs. 15 Markdown skills describe each procedure.

Table 3. Components.

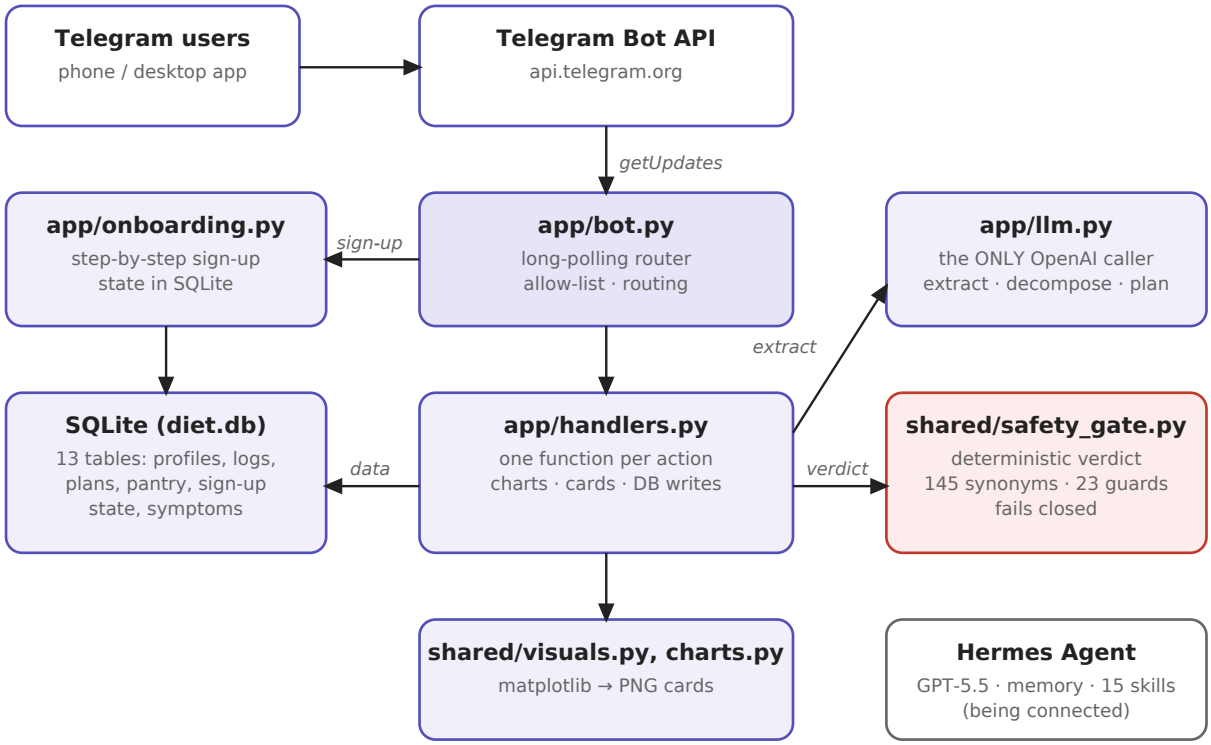


Figure 1. Deployed architecture. The red component decides safety; the language model never does.

Layer	Technology
Interface	Telegram Bot API, long polling; persistent reply keyboard and inline keyboards
Application	Python 3.12, standard-library HTTP client, matplotlib (Agg backend)
Intelligence	OpenAI models through app/llm.py (default gpt-4o); Hermes Agent 0.21.6 with GPT-5.5 for conversation
Storage	SQLite (one file, 13 tables)
Operations	Ubuntu 24.04, systemd service, dedicated unprivileged user

Table 4. Technology stack.

4. How it works: the workflows

4.1 Life of a message

Every interaction starts as a Telegram *update*. bot.py fetches updates with long polling and handles them one by one:

- **Allow-list.** If ALLOWED_TELEGRAM_IDS is set and the sender is not on it, the update is ignored.
- **Sign-up first.** If the user is in the middle of signing up, text goes to the current question; commands and grid buttons re-ask that question instead of starting something else; /start always works.
- **Grid labels.** A tap on “Today” arrives as text; the router strips it to the word “today” and treats it as the /today command (typing “today” works too).
- **Dispatch.** Commands and inline-button callbacks go to their handler; photos go to the safety check; free text is matched for intents (“can I eat”, “tired”, “plan”) and otherwise treated as a food check (to be replaced by the Hermes chat bridge, Section 16).

- **Reply.** Handlers write to the database, render PNG cards with matplotlib and send them with sendPhoto.

4.2 Sign-up

New users are interviewed one question at a time (Figure 2). Each answer is validated and saved before the next question is asked, so a restart never loses progress and an answered question is never asked again. Taps on allergen buttons toggle them (a check mark shows what is selected); other allergens can be typed, e.g. “kiwi, mustard”. Numbers are checked against human ranges (age 13–100, height 120–230 cm, weight 30–300 kg). At the end the calorie and macro targets are computed and the profile card is sent.

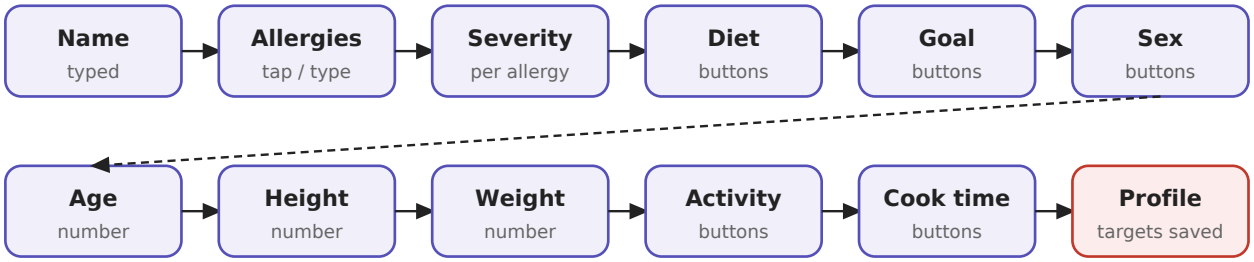
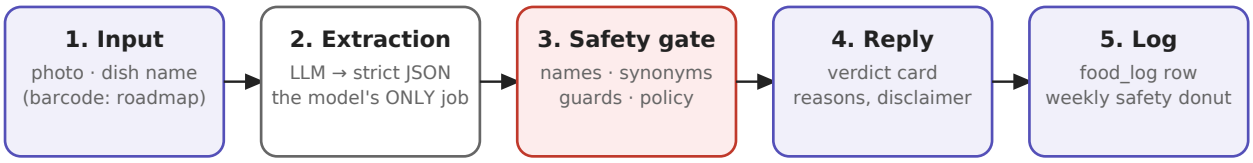


Figure 2. The sign-up conversation: 11 questions, then the profile is saved.

4.3 Food safety check



Same input → same verdict, every time. Uncertainty is never reported as SAFE.

Figure 3. The safety-check pipeline. Only step 2 involves a language model.

The extraction is a strict JSON object, for example:

```
{ "image_type": "ingredient_label", "readability": 0.95,
  "identified_ingredients": ["sugar", "whey powder", "cocoa"],
  "likely_hidden_ingredients": [], "precautionary_warnings": ["may contain peanut"] }
```

For a dish name the model lists typical and likely-hidden ingredients (tiramisu: mascarpone, eggs, ladyfingers, cream, coffee). Labels in any language are transcribed and translated by the extraction prompt before the gate runs. Section 5 describes the gate itself.

4.4 Weekly plan and shopping list

/plan sends the profile and pantry to the planner model, which returns seven days of recipes. Every recipe is re-screened by the safety gate before it enters the schedule; unsafe ones are marked for regeneration. The recipes and the schedule are stored, and the shopping list is computed as *ingredients required minus pantry stock*, with exact quantities, grouped by store section, and sent as a card.

4.5 Tracking and reports

Meals logged in food_log feed the Today dashboard, the weekly bar chart and the 30-day trend (with weight from weight_log on a second axis). /report sends the week, the month and a donut of the week's safety checks.

5. The deterministic safety gate

The gate (shared/safety_gate.py) turns an extraction and a profile into one of five verdicts. It works in four stages.

5.1 Resolving the user's allergies

Users write allergies in their own words. The gate maps them to the 14 canonical allergens of EU Regulation 1169/2011 Annex II and the US FDA “Top 9”: celery, egg, fish, lupin, milk, mollusc, mustard, peanut, sesame, shellfish, soy, sulphite, tree nut and wheat. Aliases are handled (dairy and lactose → milk; gluten → wheat; soya → soy; sulfite → sulphite; seafood → shellfish, mollusc and fish), plurals are handled (“Peanuts”), and an ambiguous word maps to every allergen it could mean (“nuts” → tree nut *and* peanut). If two entries map to the same allergen, the strictest severity wins. An allergen with no synonym list, such as kiwi, is matched by its literal name and can never produce SAFE.

5.2 Matching label terms

Allergen	Examples of label terms that trigger it (145 in total)
milk	whey, casein, caseinate, lactose, lactalbumin, butter, E966, lactitol
egg	albumin, albumen, ovalbumin, lysozyme, mayonnaise, E1105
fish	anchovy, Worcestershire sauce, fish sauce, surimi, caviar
soy	lecithin, E322, tamari, miso, edamame, natto
wheat	gluten, semolina, spelt, durum, bulgur, seitan, farro, kamut, barley, rye, malt
sulphite	E220–E228 (exact match), metabisulphite

Table 5. Sample of the synonym table.

Terms match as substrings (“whey” in “whey powder”) or exactly (E-numbers). Substring matching alone would raise false alarms, so **23 false-positive guards** skip known traps: coconut, almond, oat, rice and soy milk; cocoa and shea butter; peanut butter for milk; nutmeg, water chestnut and coconut for tree nuts; buckwheat for wheat; eggplant for egg; and “-free” labels.

5.3 Verdict policy

#	Condition (checked strictest first)	Verdict
1	Input unusable: not food, unreadable, or label readability below 0.85	CAN'T VERIFY (treat as not safe)
2	A declared allergen is in the ingredient list	DO NOT EAT
3	A “may contain” trace and the user's severity is severe or anaphylactic	DO NOT EAT
4	A trace or likely-hidden allergen at lower severity	CAUTION
5	The user has an allergen without a synonym list	CAUTION (never SAFE)
6	Photo of a cooked dish and a severe allergy	CAUTION — a photo cannot prove absence
7	Photo of a cooked dish otherwise	LIKELY SAFE
8	Complete, readable label with no match	SAFE

Table 6. Verdict policy.

5.4 A worked example

A user with a moderate fish allergy asks about a beef stew whose label lists “beef, Worcestershire sauce”. The extraction contains both terms; the gate normalises them, finds the synonym “worcestershire” under fish, checks that no guard applies, and returns DO NOT EAT with the reason “worcestershire sauce → fish (severity: moderate, found in: ingredients)”. Every DO NOT EAT and CAN'T VERIFY reply also tells the user to use emergency medication and call emergency services in case of a reaction.

6. Feature catalogue

NutriGent was designed as 48 features. Each is marked **working** (implemented in the bot's code and exercised by tests or the end-to-end run), **partial** (part works, or the Hermes agent performs it from a skill rather than tested code) or **roadmap** (designed, not built). Totals: 18 working, 8 partial, 22 roadmap. The same catalogue is kept in FEATURES.md in the repository.

6.1 Safety

#	Feature	Status	What exists today
F1	Photo safety check	working	Photo → extraction → gate → verdict card (real reading needs an OpenAI key)
F2	Barcode check (Open Food Facts)	roadmap	Procedure in a skill; no code path yet
F3	Text safety check	working	Dish decomposed into typical and hidden ingredients
F4	Hidden-allergen synonyms	working	145 terms for 14 allergens
F5	False-positive guards	working	23 patterns
F6	Per-allergen severity policy	working	Severity decides how traces are treated
F7	Fail-closed on unreadable input	working	Readability < 0.85, unreadable, not food → CAN'T VERIFY
F8	Restaurant menu scan	partial	Agent skill; the bot's menu button is not wired
F9	Allergy card for travel	partial	English card; local language and emergency number planned
F10	Cross-contamination guard	working	“May contain” lines handled by severity
F11	Multi-language labels	partial	Extraction prompt translates; not yet validated on real foreign labels

Table 7. Safety features.

6.2 Planning and meals

#	Feature	Status	What exists today
F12	Weekly meal plan	partial	7 dinners, gate-screened, stored, shopping list built; macro and expiry constraints only in the prompt
F13	Swap a single meal	roadmap	Button exists, no handler
F14	Leftover chaining	partial	Requested in the planner prompt; tag shown on the plan card
F15	Recipe import and make-safe	partial	Agent skill only
F16	Symptom–food correlation	roadmap	Symptoms stored; analysis not built

Table 8. Planning features.

6.3 Calorie and macro tracking

#	Feature	Status	What exists today
F17	Personal calorie target	working	Computed at sign-up (Section 8)
F18	Daily macro dashboard	working	Calorie ring and macro bars from food_log
F19	Meal logging (text / photo / barcode / voice)	partial	Text only, with a fixed 500 kcal placeholder
F20	Dynamic weekly rebalancing	roadmap	—
F21	Adaptive recalibration from weight	roadmap	Weight stored and charted

Table 9. Tracking features.

6.4 Shopping and pantry

#	Feature	Status	What exists today
F22	Pantry-aware shopping list	working	Required − pantry, exact quantities, store sections, image card
F23	Pantry sync 4× per week	roadmap	Skill written; schedule not registered
F24	Spoilage routing	roadmap	Expiry dates stored; not enforced
F25	Macro-preserving substitutions	roadmap	Described in a skill
F26	Saturday shopping reminder	roadmap	Part of the planned schedule

Table 10. Shopping features.

6.5 Daily autonomous actions

#	Feature	Status	What exists today
F27	16:00 tonight's-meal preview	roadmap	Skill written; schedule not registered
F28	17:30 full recipe delivery	roadmap	Same
F29	“Too tired” fallback	partial	Pauses tonight's meal; suggestion is fixed text, not pantry-based yet

Table 11. Autonomous actions.

6.6 Visual replies

#	Feature	Status	What exists today
F30	Verdict card	**working**	Colour band, item, reasons, disclaimer
F31	Today dashboard	**working**	Calorie ring and macro bars
F32	Weekly bar chart	**working**	7 days against target
F33	Monthly trend	**working**	30 days, 7-day average, weight axis
F34	Safety-check donut	**working**	Safe / caution / unsafe of the week
F35	Meal plan card	**working**	One row per day
F36	Shopping card	**working**	Grouped by store section
F37	Profile card	**working**	Severity-coloured allergies and targets

Table 12. Visual features.

6.7 Smart adaptations and integrations

#	Feature	Status	What exists today
F38	Batch-cooking mode	**roadmap**	Planner constraint exists, switched off
F39	Cooking-skill progression	**roadmap**	Profile field exists
F40	Eating window / fasting	**roadmap**	Profile field exists
F41	Health-flag support	**roadmap**	Profile field exists
F42	Medication–food interaction flags	**roadmap**	—
F43 –F48	Calendar, weather, to-do, notes, email, smart scale	**roadmap**	Through Hermes MCP servers; described in the life-sync skill

Table 13. Adaptations and integrations.

6.8 Added during this phase

#	Feature	Details
N1	Step-by-step sign-up	11 questions, mostly buttons, saved in onboarding_state; resumes after restarts; never loops
N2	Allergy names as people type them	dairy, lactose, gluten, nuts, plurals; fixes a fail-open defect
N3	Unknown allergens never SAFE	Literal matching and a CAUTION cap
N4	Eggplant is not egg	New guard, migrated into existing databases
N5	Button grid without slashes	Persistent keyboard; labels and typed words both work
N6	Allow-list	ALLOWED_TELEGRAM_IDS
N7	Per-user data ownership	Users can only change their own shopping items
N8	End-to-end test with a simulated Telegram	80 scripted actions through the real polling loop
N9	Hardened deployment and restored agent	systemd as an unprivileged user; Hermes with persona, memory and history

Table 14. Features added in this phase (all working).

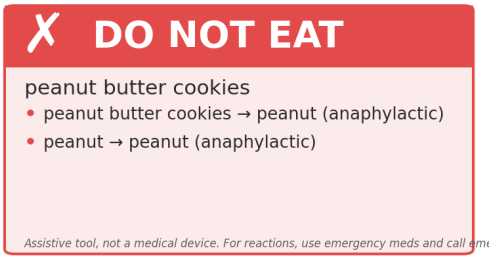
7. User interface and visual replies

NutriGent offers three ways to act, so users never need to remember syntax: a **persistent button grid** under the chat input (14 buttons labelled without slashes, e.g. “Today”), **inline buttons** attached to messages (the 12-button main menu, sign-up choices), and the classic **slash menu** registered with Telegram. Table 15 lists every action.

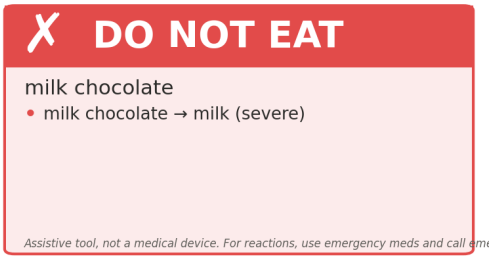
Action	Status	What happens
Start	**working**	Sign-up for new users; “Welcome back” and the menu for registered users
Menu / Help	**working**	12-button menu / short explanation
Check	**working**	Dish name or photo → verdict card (sub-buttons photo / barcode / text / menu: roadmap)
Today / Week / Month	**working**	Dashboard / weekly chart / monthly trend
Plan	**partial**	7-day plan card and shopping card (F12)
Shopping	**working**	Shopping card
Tonight	**partial**	Shows the daily dashboard; tonight's meal with Cooking / Swap / Recipe buttons is planned
Log	**partial**	Text log with placeholder calories (F19)
Tired	**partial**	Pauses tonight's meal (F29)
Symptom	**working**	“/symptom bloating after lunch” is stored (picker buttons: roadmap)
Report	**working**	Week chart, month chart and safety donut
Card	**working**	Allergy card text for restaurants
Profile	**working**	Profile card (editing: roadmap)

Table 15. User actions.

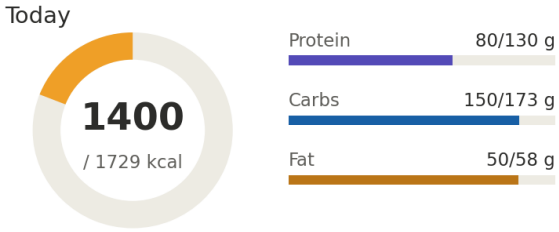
Structured answers are images rendered with matplotlib from real database rows and sent with sendPhoto, using one colour language: green for on target or safe, amber for under target or caution, red for over target or unsafe, purple for the brand and target lines, grey for unverified or no data. Figures 4–6 are images the bot produced during the end-to-end test for test users with fictional data.



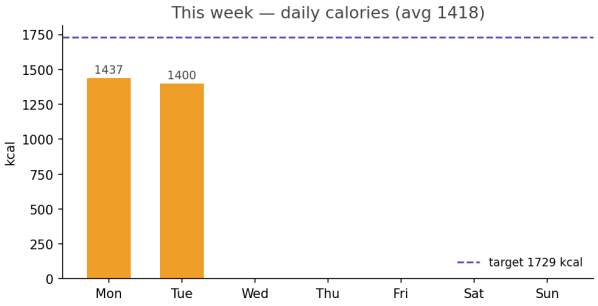
Verdict: peanut butter cookies, anaphylactic peanut allergy



Verdict right after sign-up: milk chocolate, severe milk allergy

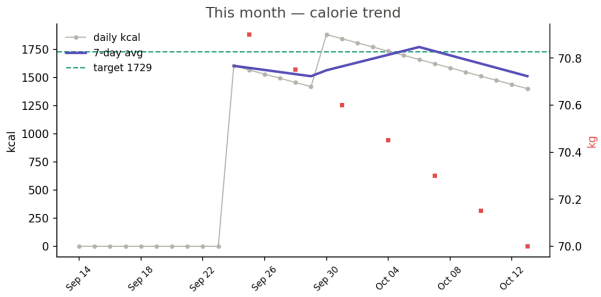


Today: calorie ring and macro bars

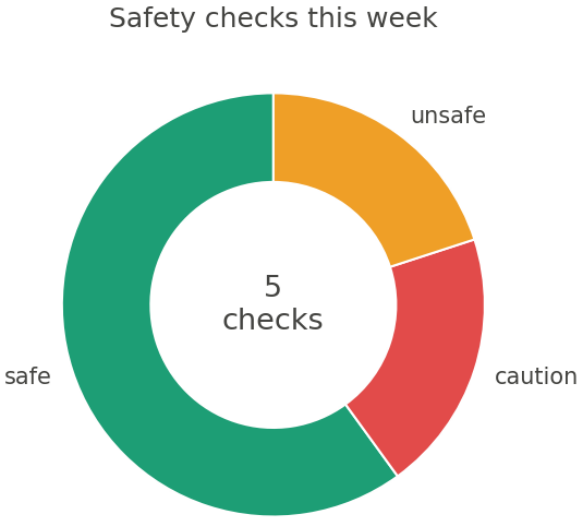


Week: daily calories against target

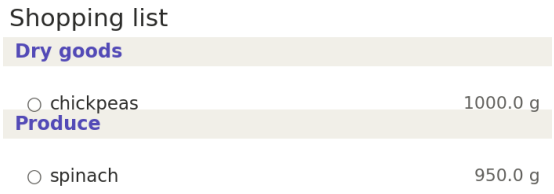
Figure 4. Verdict cards and daily/weekly tracking.



Month: 30-day trend, 7-day average, weight



Weekly safety-check donut



Shopping list grouped by store section



Profile card created by the sign-up flow

Figure 5. Trend, safety, shopping and profile cards.

8. Nutrition model

Daily energy targets use the Mifflin–St Jeor equation, the most widely used predictive equation for resting energy expenditure:

```
BMR (men) = 10 × weight[kg] + 6.25 × height[cm] − 5 × age + 5
BMR (women) = 10 × weight[kg] + 6.25 × height[cm] − 5 × age − 161
TDEE = BMR × activity (sedentary 1.2 · light 1.375 · moderate 1.55 · active 1.725 · athlete 1.9)
Target = TDEE × 0.80 (lose) · 1.00 (maintain) · 1.12 (gain)
Macros = 30 % protein · 40 % carbohydrate · 30 % fat (4 · 4 · 9 kcal per gram)
```

Example user	BMR	TDEE	Daily target	Protein	Carbs	Fat
Woman, 29 y, 165 cm, 60 kg, moderate, lose	1,325	2,054	1,643 kcal	123 g	164 g	55 g
Man, 25 y, 190 cm, 70 kg, moderate, gain	1,768	2,740	3,068 kcal	230 g	307 g	102 g

Table 16. Worked examples (values produced by shared/nutrition.py; both appear in the automated tests).

The “lose” deficit of 20 % and “gain” surplus of 12 % are conservative defaults. Manual overrides (for example a keto split) and weekly recalibration from the weight trend are on the roadmap (F21).

9. Data model

All state lives in a single SQLite file, which keeps the deployment simple and the data on the operator's server. New tables are created automatically at start-up, so an existing database is upgraded in place.

Table	Purpose
users	Telegram identity, name, time zone, locale
dietary_profiles	Allergies with severity, intolerances, diet, dislikes, goal, calorie and macro targets, cooking skill and time, budget, eating window, health flags
onboarding_state	Current sign-up question and answers so far (new in this phase)
allergen_synonyms	145 label terms → 14 canonical allergens (contains / exact)
synonym_exceptions	23 false-positive guards
pantry	Ingredients at home, quantity, unit, expiry date
recipes	Generated or imported recipes with macros and the gate's screening result
meal_schedule	Planned meals per day and status (planned / cooked / skipped / swapped)
shopping_list_items	Weekly list: missing quantity, store section, status, estimated cost
food_log	Everything eaten or checked: source, verdict, flagged allergens, macros
preference_events	Accepted / swapped / skipped / loved / hated recipes, for personalisation
weight_log	Weight over time
symptom_log	Symptoms with severity, for future food–symptom correlation

Table 17. Database schema (shared/schema.sql).

10. The agent layer: Hermes Agent and skills

Hermes Agent provides what deterministic code cannot: natural conversation in several languages (pilot users write in English and Persian), a persona, long-term memory of each user, vision, and a scheduler. NutriGent's behaviour is taught to the agent through a system prompt (AGENT_PROMPT.md) and 15 *skills* — Markdown procedures that tell the agent when to act and which project code to use:

Skill	Trigger	Job
onboard-user	/start, new user	Interview → profile → menu
food-safety-check	photo, “can I eat X?”	Extract → gate → verdict card
charts-dashboard	/today /week /month	food_log → chart → photo
weekly-plan	Sunday 09:00, “plan my week”	Profile + pantry → plan → re-screen → card
shopping-assistant	/shopping, Saturday 18:00	Pantry diff → grouped list → reminder
daily-nudge	16:00 / 17:30 / 21:30	Preview → recipe → follow-up
log-meal	/log, plate photo, voice	Estimate macros → food_log → dashboard
handle-friction	“tired”, “sold out”	Fallback meal or substitution + rebalance
pantry-sync	Mon/Wed/Fri/Sun 23:00	Reconcile → subtract → expiry flags → memory
weekly-report	Sunday 20:00	Aggregate → 3 charts → correlation → recalibrate
update-profile	“I'm also allergic to…”	Patch profile → re-screen the week
eating-out	menu photo, /card	Per-dish gate → safe / ask / avoid
import-recipe	recipe URL, “make safe”	Extract → screen → substitute → rescale
life-sync	calendar / weather / todo / email	Integrations via MCP servers
visual-output	every structured reply	Convention: answer with images

Table 18. The 15 skills.

The agent is installed on the server, logged in to OpenAI (GPT-5.5) and restored from the previous server's backup with its persona, long-term memory and 947 messages of history. A smoke test confirmed that memory survived the move: asked for its name and a stored user's allergies, it answered correctly in 14 seconds. Its own Telegram gateway is deliberately disabled, because only one program may read a bot's updates; connecting it behind the bot is the main item of roadmap phase 2. The planned weekly schedule is:

When	Action
Sunday 09:00	Weekly plan and shopping list
Daily 16:00 / 17:30 / 21:30	Tonight's preview, full recipe, one gentle follow-up
Saturday 18:00	Shopping reminder for pending items
Sunday 20:00	Weekly report: three charts and symptom correlation
Mon / Wed / Fri / Sun 23:00	Pantry sync and distilled memory update

Table 19. Planned autonomous schedule (roadmap phase 4).

11. Engineering process and fixes in this phase

Work in this phase followed a test-first discipline: every behaviour change started with an automated test, which was run and seen to fail for the expected reason before the code was changed, and the whole suite was re-run before every deployment. The phase began with a review of the full code base and of the previous server's backup.

11.1 A fail-open defect in allergen matching

The gate recognised only the canonical allergen names. A user who wrote “dairy”, “peanuts”, “gluten” or “nuts” matched nothing, so every food was reported SAFE — a direct violation of goal G1. The alias resolution of Section 5.1 fixed it.

User wrote	Label contains	Before	After
dairy	milk powder	SAFE	DO NOT EAT
lactose	whey	SAFE	DO NOT EAT
Peanuts	peanut butter	SAFE	DO NOT EAT
gluten	wheat flour	SAFE	DO NOT EAT
nuts	almonds / peanut oil	SAFE	DO NOT EAT
kiwi (no synonym list)	kiwi fruit / water, salt	SAFE / SAFE	DO NOT EAT / CAUTION
egg	eggplant	DO NOT EAT (false alarm)	SAFE

Table 20. Behaviour before and after the fix (all cases are automated tests).

11.2 The sign-up loop

During pilot testing on a phone, every message from a new user produced the same welcome and the question “what should I call you?”. The cause was structural: sign-up had no memory of which question had been asked, so each answer was treated as a fresh request from a user without a profile. A failing test reproduced exactly the observed messages; the fix was the persisted sign-up conversation of Section 4.2.

11.3 Access control and security

- The previous server let *any* Telegram user talk to the agent, which can run shell commands, and ran it as root. The new deployment runs as an unprivileged user, disables open access and keeps the agent's Telegram connection off.
- The bot has an allow-list of Telegram IDs.
- A user could mark another user's shopping item as bought by guessing its number; ownership is now checked.
- Secrets and all databases (which contain health data) are excluded from version control; a script that contained a real user's health profile was removed from the repository.

11.4 Interface

The persistent button grid now shows clean labels without slashes; the router maps both the label and the typed word to the right command, so the grid behaves like a native app menu.

12. Testing and evidence

12.1 Automated test suites

Suite	What it checks	Result
test_safety_gate.py	21 fixtures: direct and hidden allergens, traces by severity, unreadable labels, cooked dishes, false-positive guards, E-numbers, typed allergy names, unknown allergens	21 / 21
test_onboarding_flow.py	Full sign-up with stored profile and targets; answering the name moves on (loop regression); bad numbers re-asked; no-allergy path; commands mid-sign-up; returning user	6 / 6
test_reply_keyboard.py	Grid labels have no slash; grid is persistent; every label runs its command; typed words work; /start sends the grid	5 / 5
test_access.py	Allow-list blocks and admits correctly; shopping-item ownership	4 / 4
test_onboarding_buttons.py	Sign-up button layouts and callback formats	2 / 2
test_bot_flow.py	10 simulated updates through router, handlers and image rendering	all run

Table 21. Automated tests: 38 tests plus a flow script; all pass on macOS and on the server.

12.2 End-to-end test against a simulated Telegram

Unit tests replace the network with stubs, so they cannot show what a user actually experiences. A fake Telegram Bot API was therefore written (tests/e2e_fake_telegram.py): a small HTTP server on 127.0.0.1 implementing getUpdates, sendMessage, sendPhoto, getFile and answerCallbackQuery. Like the real service it rejects messages that break Telegram's HTML rules and photos that are not valid images. The *unmodified* polling loop of bot.py is pointed at it and driven through 80 scripted actions: 19 slash commands (one deliberately unknown), 29 button taps, 2 photos, 8 free-text messages including Persian, a voice note, and a complete 21-step session by a brand-new user. It runs in about 20 seconds with a peak of about 120 MB of memory, with identical results on a laptop and on the server.

Outcome	Count	Details
Correct reply	66	All commands, main-menu buttons, photo and text checks, charts, shopping, report, card, profile, the full sign-up and the returning-user greeting
No reply	13	Inline buttons without a handler yet: check options (4), log options (3), symptom picker (2), tonight's meal actions (3), “unavailable”
Rejected by Telegram	1	“mac & cheese”: an unescaped “&” in the reply

Table 22. End-to-end results.

The new user's session from that run, verbatim except for shortening:

User sends	NutriGent replies
/start	Hi, I'm NutriGent — your personal diet and allergy assistant … First, what should I call you?
Nima	Nice to meet you, Nima! Do you have any food allergies? Tap all that apply …
[taps Milk] · types “kiwi”	Your allergies: milk. … · Your allergies: milk, kiwi. Tap more, or Done.
[taps the Today grid button]	Let's finish setting you up first. … (same question again)
[Done] · [Severe] · [Mild]	How severe is your milk allergy? · … kiwi allergy? · Do you follow a particular diet?
[Halal] · [Gain] · [Male]	What's your main goal, Nima? · … male or female? · How old are you?

User sends	NutriGent replies
abc	Please send a number between 13 and 100. How old are you?
25 · 190 cm · 70kg	Your height in cm? · Your weight in kg? · How active are you on a normal week?
[Moderate] · [15 min]	How long can you usually spend cooking? · [profile card] You're set. Plan your week?
/today	[dashboard] Today: 0/3068 kcal
/start	Welcome back, Nima! Tap a button below.
/check milk chocolate	DO NOT EAT • milk chocolate → milk (severity: severe, found in: ingredients)

Table 23. Transcript of the sign-up in the end-to-end run.

Inspecting replies and images by hand found problems that an automatic “did it reply?” check cannot see; they are listed in Section 15.

13. Deployment

The pilot runs on an Ubuntu 24.04 virtual server with 1 vCPU and 2 GB of RAM, shared with an unrelated project. Because the bot uses long polling, no inbound port is opened.

Aspect	Choice
Isolation	Dedicated unprivileged system user; code, virtual environment and data under /opt/nutrigent
Process management	systemd service: starts on boot, restarts on failure, memory capped at 400 MB
Hardening	NoNewPrivileges, ProtectSystem=strict, PrivateTmp, ProtectHome, write access only to the data directory
Secrets	Bot token and allow-list in a .env file readable only by the service user, entered by the operator
Database upgrades	Idempotent: new tables and guards are created at start-up; user data is preserved
Agent	Hermes Agent 0.21.6 for the same user; profile restored from backup; Telegram gateway disabled
Release procedure	Run all suites locally → copy the committed tree → run all suites on the server → restart → check logs
Resources	Bot about 70 MB idle and 120 MB peak; whole server about 0.5 GB of 2 GB in use

Table 24. Deployment configuration (deploy/nutrigent-bot.service in the repository).

14. Privacy, safety and ethics

Data	Where it lives	Who sees it
Profile: allergies, goals, biometrics	Local SQLite on the operator's server	The user and the server operator
Food, weight and symptom logs; pantry; plans	Local SQLite	The user and the server operator
Agent memory and chat history	Local Hermes files on the server	The user and the server operator
Photo sent for a safety check	Sent to the model provider for extraction	The provider, under its data policy
Allergen names (context for extraction)	Sent with the photo	The provider, under its data policy
Name, Telegram ID, history	Never sent to the extraction API	Stays on the server

Table 25. Where data lives.

- NutriGent is an assistive tool, **not a medical device**. It does not diagnose or treat conditions, replace a dietitian, allergist or doctor, or guarantee allergy safety — it reduces risk. This is stated at sign-up and on every DO NOT EAT or CAN'T VERIFY verdict.
- Reported reaction symptoms trigger immediate guidance to use emergency medication and call emergency services.
- Uncertainty is treated as risk: unreadable labels, unknown allergens and cooked dishes never receive a plain SAFE for severe allergies.
- Access is restricted, services run without root privileges, and secrets and health data are kept out of version control.

15. Current limitations

- **13 inline buttons have no handler** (Table 22).
- **Free text** that is not a recognised intent is treated as a food check (“hello” produces a CAUTION card) until the agent is connected behind the bot.
- **Placeholder logic:** meal logging records a fixed 500 kcal; the “too tired” suggestion is static; “Tonight” shows the dashboard instead of tonight's meal.
- **Meal plans for severe allergies:** planned recipes are screened as if they were photos of cooked dishes, so rule 6 flags every meal (Figure 6). Recipes have known ingredient lists and should be screened as labels.
- **Text escaping:** characters such as “&” in user text can make Telegram reject a reply.
- **Synonym coverage:** 145 terms are a starting set that needs expansion and review by an allergy expert.
- **Model dependence:** without an OpenAI API key, photo reading and planning use fixed sample answers.
- **Scale:** SQLite and long polling suit a 3–10 user pilot; larger use needs PostgreSQL and webhooks.
- Crowd-sourced product data (planned barcode feature) can be stale and must be cross-checked with the label.

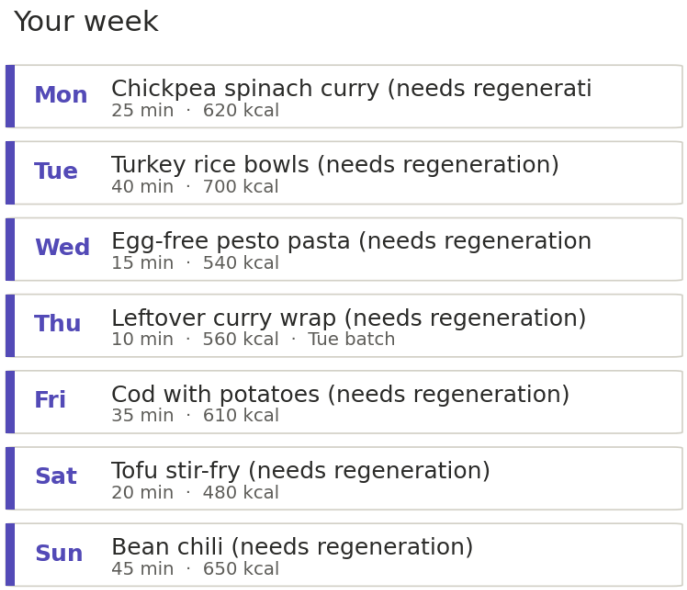


Figure 6. A generated plan for a user with an anaphylactic allergy: every meal is flagged “needs regeneration” because of the screening issue described above.

16. Roadmap

Phase	Scope	Status
1. Safety core and pilot	Deterministic gate, synonyms and guards, cards and charts, sign-up, button grid, access control, tests, server deployment, agent restored	done
2. Core flows	Engine that returns replies (testable without Telegram) and a CLI for the agent; bot as front door with a Hermes bridge for free chat (per-user sessions, “typing…”, fallback when the agent is down); wire the 13 silent buttons; real meal-logging estimates; HTML escaping; screen planned recipes as labels; real “Tonight”; pantry-based “Too tired”	next — design agreed
3. Better inputs	Barcode → Open Food Facts (F2); menu scan in the bot (F8); swap one meal (F13); photo and voice logging (F19)	planned
4. Autonomy	Scheduled jobs that call the CLI: daily nudges, Sunday plan and report, Saturday reminder, pantry sync, preference memory (F23, F26–F28)	planned
5. Insights and adaptation	Symptom–food correlation (F16), weekly rebalancing (F20), weight-based recalibration (F21), profile editing, Persian interface	planned
6. Integrations and scale	Calendar, weather, to-do, notes, email, smart scale (F43–F48); health flags and medication information (F41–F42); PostgreSQL and webhooks	planned

Table 26. Roadmap.

Phase 2 restructures the code so that the agent and the deterministic engine cooperate without giving up testability (Figure 7). Every action will return a *Reply* (text, images, buttons) instead of sending it, so it can be tested without Telegram and reused by any front end. The bot stays the only holder of the bot token and routes commands, buttons and photos to the engine and open conversation to Hermes through a small bridge using the agent's one-shot mode with one session per user. Hermes skills will call one tested command-line action each instead of improvising code, so the agent's actions become as reliable as the engine's.

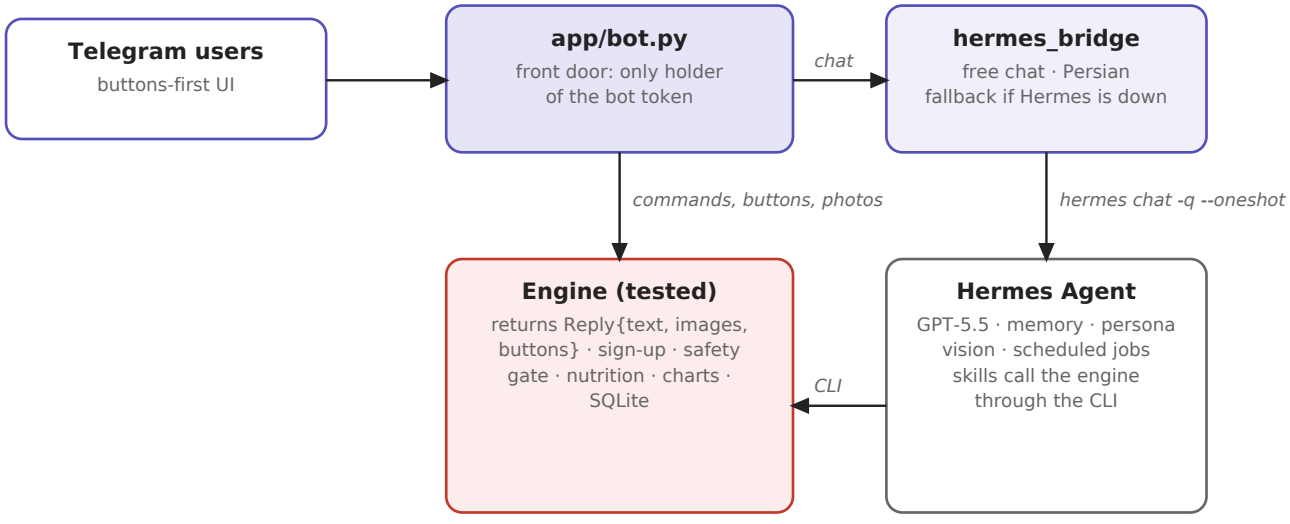


Figure 7. Target architecture after phase 2.

17. Conclusion

NutriGent shows that a language-model agent can be useful for everyday diet and allergy management when it is kept away from the final safety decision. Separating extraction (model) from decision (deterministic gate) made the system auditable and testable — which is exactly what allowed this phase to find a serious fail-open defect, prove it with tests and fix it. The pilot is deployed: users can sign up step by step, check foods by photo or name, receive visual dashboards, plans and shopping lists, and the safety core is covered by automated tests. The end-to-end test measured honestly what remains: a set of buttons, placeholder logic in logging and fallback meals, and the link between the bot and the conversational agent. These form the agreed next phase, after which the autonomous schedule, better inputs and adaptive features complete the original design.

Appendix A. Repository layout

app/ — bot.py (router) · onboarding.py (sign-up) · handlers.py (actions) · llm.py (OpenAI adapter)
shared/ — safety_gate.py · nutrition.py · visuals.py · charts.py · reporting.py · db.py · telegram_io.py · telegram_menu.py · schema.sql · allergen_seed.sql · prompts/
skills/ — 15 Hermes skills · tests/ — unit suites and the end-to-end test · deploy/ — systemd unit and agent wrapper · docs/ — this report · README.md · FEATURES.md · AGENT_PROMPT.md · requirements.txt · .env.example

Appendix B. Running the system

```
python3 -m venv .venv && . .venv/bin/activate && pip install -r requirements.txt
cp .env.example .env # TELEGRAM_TOKEN, optional OPENAI_API_KEY, ALLOWED_TELEGRAM_IDS
set -a; . ./.env; set +a; python3 app/bot.py

python3 tests/test_safety_gate.py
python3 tests/test_onboarding_flow.py
python3 tests/test_reply_keyboard.py
python3 tests/test_access.py
python3 tests/test_onboarding_buttons.py
python3 tests/test_bot_flow.py
python3 tests/e2e_fake_telegram.py
```